

Project Test Report: AnomalyDetection

This report reproduces the examples and experiments of the AnomalyDetection project by following the step-by-step instructions in the [README](#). Every outcome was measured on two platforms — an RTX 4090 workstation and an A100 server — to verify reproducibility.

Outline

- [1. Hardware and Software Specifications](#)
- [2. Evaluation Reproduction](#)
 - [2.1 Examples](#)
 - [2.2 Technical and Overhead Metrics](#)
 - [2.2.1 Technical Metrics: Image Level](#)
 - [2.2.2 Technical Metrics: Pixel Level](#)
 - [2.2.3 Technical Metrics: Instance Level](#)
 - [2.2.4 Overhead Metrics: Compute](#)
 - [2.2.5 Overhead Metrics: Memory](#)
 - [2.3 Analysis](#)
- [3. Extensibility](#)
 - [3.1 Adding a New Dataset](#)
 - [3.2 Adding the YOLO 26 Model](#)
- [4. Discussion](#)

1. Hardware and Software Specifications

Configurations and results are compared across two platforms:

- Small Machine: RTX 4090 workstation
- Full Machine: A100 server

Table 1 lists their hardware and software.

Table 1: Hardware and Software Configuration

Subsystem	Small Machine	Full Machine
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04.5
CPU	Intel Core i9-13900K	2 × Intel Xeon Gold 6430
System Memory	32 GB	256 GB
GPU & VRAM	NVIDIA RTX 4090 24 GB	NVIDIA A100 80GB
Python	3.12	3.12
PyTorch	2.14.0 (+cu130)	2.14.0 (+cu130)
CUDA Toolchain	Driver 535.54.03, CUDA 12.8	Driver 550.144.03, CUDA 12.4

2. Evaluation Reproduction

2.1 Examples

The environment was set up per [Environment Setup](#), then the test cases under [Examples](#) were reproduced. Only the Full Machine results are shown.

Following Steps 1 to 4, the target model was selected and loaded:

The screenshot shows a three-panel interface for model setup. The left panel, titled 'Model setup', contains dropdowns for 'Task type' (Anomaly detection), 'Application domain' (textile), and 'Model' (PatchCore - ZJU-Leaper (normal only)). Below these is a 'Ready' status box with details: Method: PatchCore, Application domain: textile, Training corpus: zju-leaper, Training split: normal_only, and Weight: PatchCore.ckpt. At the bottom are buttons for 'Load model', 'Unload model', 'Inspect checkpoint', and a large orange 'Run detection' button. The middle panel, 'Selected dataset image', is empty with a message 'No image loaded yet.' and navigation buttons '← Previous' and 'Next →'. The right panel, 'Detection result', is also empty.

Following Step 5, the image to be evaluated was selected and loaded:

The screenshot shows a 'Dataset & sample selection' interface. It includes dropdowns for 'Dataset' (ZJU-Leaper) and 'Texture / pattern' (Pattern 1-4 (Train patterns)). There are radio buttons for 'Split' (test selected) and 'train'. A 'Random images' section shows a slider from 4 to 12, currently set at 8, with a 'Load random images' button. Below are dropdowns for 'Image selection' (Defect only) and 'Sample regime' (Full-shot selected, Few-shot unselected). A status bar at the bottom indicates: 'Loaded 8 images — Defect only, Full-shot — from ZJU-Leaper / Pattern 1-4 (Train patterns) / test.'

Following Step 6, anomaly detection was executed successfully. The resulting output confirms that the platform and its configuration are working correctly, so the remaining tests can be performed.



2.2 Technical and Overhead Metrics

Following the instructions in [README §4.1.3 Benchmarking](#), the complete benchmark suite was run on both machines. The resulting data are reported below.

Both platforms used the same configuration:

Item	Value
Dataset	ZJU-Leaper test split
Samples	350 in total
Input size	640 × 640
Precision	fp32
Metrics	Image, pixel, instance and overhead
Evaluated configurations	19

The 19 configurations are 10 anomaly models, 6 detection models and 3 segmentation models. Each appears only in the tables its output supports — GANomaly, for instance, produces no spatial anomaly map and is therefore absent from the pixel-level table. Full data: [Small Machine](#), [Full Machine](#).

2.2.1 Technical Metrics: Image Level

This part evaluates the image-level metrics defined by [README §3.3.1](#):

- **Image AUROC:** how often a defective image scores above a normal one. 0.5 is random guessing, 1.0 is perfect, and it needs no decision threshold.
- **Image Precision:** of the images flagged as defective, the share that really are defective.
- **Image Recall:** of the truly defective images, the share that is flagged.
- **Image F1:** one score that balances Precision against Recall, and drops if either of them is poor.

Key observation: The two machines agree on Image AUROC to within **0.2 points** for all ten anomaly detectors, while the thresholded metrics, which depend on the calibration of the decision threshold, drift by up to 4.4 points.

Table 2: Image-level testing results on Small Machine

Model	Image AUROC	Image F1	Image Precision	Image Recall
Dinomaly	0.9797	0.9320	0.9505	0.9143
EfficientAD	0.9502	0.8586	0.9535	0.7810
GANomaly	0.6548	0.5159	0.3876	0.7714
MoECLIP	0.9877	0.9314	0.9596	0.9048
PaDiM	0.8282	0.7138	0.5854	0.9143
PatchCore	0.9932	0.9474	0.9519	0.9429
Reverse Distillation	0.6204	0.5455	0.3941	0.8857
STFPM	0.9497	0.8531	0.8491	0.8571
SuperSimpleNet	0.5847	0.5000	0.3699	0.7714
WinCLIP	0.9772	0.9026	0.9778	0.8381

Table 3: Image-level testing results on Full Machine

Model	Image AUROC	Image F1	Image Precision	Image Recall
Dinomaly	0.9797	0.9151	0.9065	0.9238
EfficientAD	0.9501	0.8557	0.9326	0.7905
GANomaly	0.6548	0.4802	0.3244	0.9238
MoECLIP	0.9877	0.9209	0.9000	0.9429
PaDiM	0.8284	0.7138	0.5854	0.9143
PatchCore	0.9932	0.9469	0.9608	0.9333
Reverse Distillation	0.6222	0.5378	0.3810	0.9143
STFPM	0.9497	0.8458	0.8854	0.8095
SuperSimpleNet	0.5852	0.4598	0.3003	0.9810
WinCLIP	0.9772	0.8584	0.8017	0.9238

2.2.2 Technical Metrics: Pixel Level

This part evaluates the pixel-level metrics defined by [README §3.3.2](#):

- **Pixel AUROC:** how often a defective pixel scores above a normal pixel, pooled over the pixels of all images. 0.5 is random guessing.

- **Pixel AUPRO:** how much of each defective region is covered before the false-positive rate passes a fixed limit. Every connected region counts equally, so a small defect is not hidden by a large one.
- **Pixel F1:** how well the predicted defect pixels match the true ones at a fixed pixel threshold, penalised both by missed pixels and by extra pixels.
- **Pixel IoU:** overlap between the predicted defect area and the true defect area, as intersection over union.
- **IAP (Instance Average Precision):** how well defect regions are found, averaged over connected regions with each region weighted equally.

Key observation: Both machines rank MoECLIP first and SuperSimpleNet last, but their absolute pixel-level values do not overlap; see [§2.3](#).

Table 4: Pixel-level testing results on Small Machine

Model	Pixel AUROC	Pixel AUPRO	Pixel F1	IAP
Dinomaly	0.7452	0.6029	0.1470	0.0574
EfficientAD	0.6072	0.4814	0.1035	0.0287
MoECLIP	0.9039	0.8374	0.5447	0.3983
PaDiM	0.7559	0.7117	0.1282	0.0528
PatchCore	0.6730	0.5925	0.0841	0.0373
Reverse Distillation	0.3529	0.5415	0.4321	0.2120
STFPM	0.6807	0.6353	0.0954	0.0546
SuperSimpleNet	0.4645	0.4108	0.0548	0.0184
WinCLIP	0.7483	0.7399	0.1482	0.1240

Table 5: Pixel-level testing results on Full Machine

Model	Pixel AUROC	Pixel AUPRO	Pixel F1	Pixel IoU
Dinomaly	0.9805	0.8895	0.4247	0.6293
EfficientAD	0.7863	0.5437	0.4814	0.5549
MoECLIP	0.9857	0.9701	0.6733	0.7006
PaDiM	0.9521	0.9192	0.1658	0.3531
PatchCore	0.9833	0.9136	0.4005	0.6259
Reverse Distillation	0.6986	0.7003	0.0263	0.0868
STFPM	0.9570	0.8744	0.3461	0.5682
SuperSimpleNet	0.4547	0.5033	0.0252	0.0792
WinCLIP	0.9248	0.9140	0.1080	0.3078

The Small Machine run did not measure `Pixel IoU`, so Table 4 omits that column. Both tables cover the 9 anomaly models that emit per-pixel maps; GANomaly is reported in the image-level and overhead tables only.

2.2.3 Technical Metrics: Instance Level

This part evaluates the instance-level metrics defined by [README §3.3.3](#):

- **AP**: how well the detected boxes follow the precision–recall trade-off, averaged over IoU thresholds 0.50–0.95 (COCO style). Higher is better.
- **AP50 / AP75**: the same measure at one IoU threshold. 0.50 accepts a loosely placed box, 0.75 demands a tight one. IoU is the overlap between a predicted box and the true box.
- **Precision**: of the boxes reported as defects, the share that overlaps a real defect ($\text{IoU} \geq 0.50$).
- **Recall**: of the real defects, the share that a reported box covers.
- **F1**: one score that balances box Precision against box Recall.
- **TP / FP / FN**: the counts behind those numbers — correctly found defects, spurious boxes, and missed defects.

Key observation: The thresholded **F1 reproduces exactly** across the two machines, while AP50 is higher on the Full Machine.

Table 6: Instance-level testing results on Small Machine

Model	AP50	AP	Precision	Recall	F1	TP	FP	FN
Cascade R-CNN	0.6232	0.3267	0.6327	0.6813	0.6561	124	72	58
DETR	0.0000	0.0000	0.0000	0.0000	0.0000	0	0	182
Faster R-CNN	0.6247	0.3185	0.5478	0.6923	0.6117	126	104	56
YOLO11n	0.4555	0.2316	0.9091	0.3297	0.4839	60	6	122
YOLOv8n	0.4595	0.2227	0.8571	0.3626	0.5097	66	11	116
YOLOv8s	0.4332	0.1991	0.9286	0.2143	0.3482	39	3	143

Table 7: Instance-level testing results on Full Machine

Model	AP50	AP	AP75	Precision	Recall	F1	TP	FP	FN
Cascade R-CNN	0.6428	0.3333	0.3042	0.6327	0.6813	0.6561	124	72	58
DETR	0.0022	0.0011	0.0001	0.0000	0.0000	0.0000	0	0	182
Faster R-CNN	0.6623	0.3321	0.3014	0.5502	0.6923	0.6131	126	103	56
YOLO11n	0.5754	0.2846	0.2583	0.9091	0.3297	0.4839	60	6	122
YOLOv8n	0.5861	0.2721	0.2130	0.8571	0.3626	0.5097	66	11	116
YOLOv8s	0.5380	0.2512	0.1866	0.9286	0.2143	0.3482	39	3	143

- Precision, Recall, F1 and TP/FP/FN are taken at confidence threshold 0.25. The Small Machine did not measure AP75, so Table 6 omits that column.
- A standalone `adh evaluate` run on the DETR checkpoint independently confirmed `TP=0` and `FP=0`.
- Segmentation reproduces as well: UNet++ 0.6833 / DeepLabV3+ 0.6479 / Mask R-CNN 0.7539 on the Small Machine (Dice), and 0.6833 / 0.6477 / 0.7242 on the Full Machine (Mask AP50).

2.2.4 Overhead Metrics: Compute

This part evaluates the compute metrics defined by [README §3.3.5](#):

- **FPS (mean / p95):** images processed per second. The p95 is the speed that 95% of frames still reach, so it exposes occasional slow frames.
- **Latency mean / p95 / p99:** time to process one image, in milliseconds. p95 and p99 describe the slowest frames rather than the average.
- **FLOPs (G):** arithmetic work of one forward pass, in billions of operations. It is fixed by the architecture, so it is the same on any machine.
- **Wall-time:** total seconds the scored run took, accuracy pass included.
- **Max concurrent streams:** how many video streams can run at once while every frame still meets the latency budget.

Key observation: **FLOPs reproduce** for every model measured twice, while FPS and latency are host-specific and differ by 1.1×–31×.

Table 8: Compute overhead testing results on Small Machine

Model	FPS	Latency mean	FLOPs (G)	Max Concurrent Streams
EfficientAD	3.37	296.57 ms	75.91	0
Faster R-CNN	7.81	128.08 ms	268.98	0
GANomaly	10.63	94.08 ms	16.54	0
PaDiM	31.69	31.56 ms	3.69	2
PatchCore	1.39	719.00 ms	24.14	0
STFPM	22.00	45.44 ms	7.38	1
SuperSimpleNet	2.40	417.01 ms	96.70	0
UNet++	119.22	100.19 ms	250.23	0
YOLO11n	20.83	48.01 ms	6.44	0
YOLOv8n	181.69	5.50 ms	8.19	0
YOLOv8s	11.51	86.89 ms	28.65	0

Table 9: Compute overhead testing results on Full Machine

Model	FPS (mean)	FPS (p95)	Latency mean	Latency p95	Latency p99	FLOPs (G)	Wall-time
EfficientAD	3.72	3.85	268.96 ms	324.86 ms	354.60 ms	75.9065	60.7 s
Faster R-CNN	60.42	60.67	16.55 ms	18.27 ms	19.72 ms	268.9822	21.6 s
GANomaly	34.85	34.92	28.69 ms	30.37 ms	33.46 ms	33.0444	51.4 s
PaDiM	77.32	77.41	12.93 ms	13.51 ms	13.82 ms	3.6884	43.3 s
PatchCore	4.42	4.54	226.31 ms	309.07 ms	312.48 ms	24.1408	88.3 s
STFPM	74.77	81.69	13.37 ms	21.89 ms	24.02 ms	7.3767	88.5 s
SuperSimpleNet	17.38	17.44	57.53 ms	61.21 ms	71.03 ms	96.7028	93.0 s
UNet++	79.54	90.78	12.57 ms	19.50 ms	30.95 ms	250.2320	68.4 s
YOLO11n	308.48	310.60	3.24 ms	3.45 ms	3.47 ms	6.4406	11.9 s
YOLOv8n	432.21	437.37	2.31 ms	2.64 ms	2.68 ms	14.9008	11.4 s
YOLOv8s	355.33	355.92	2.81 ms	2.95 ms	3.24 ms	6.9886	11.9 s

The two runs report different metric sets: Table 8 covers `Max Concurrent Streams`, Table 9 the percentiles and the wall time.

2.2.5 Overhead Metrics: Memory

This part evaluates the memory metrics defined by [README §3.3.6](#):

- **Parameters (M):** number of learned weights, in millions. Fixed by the architecture, so it is the same on any machine.
- **Peak memory:** the most memory used at once during the run.
- **Allocator peak:** the same peak as seen by the CUDA allocator, which leaves out the Python interpreter and its libraries.
- **Retained / extra:** memory still held after the run, which is what a long-running service keeps resident.
- **Memory backend:** the counter that produced the peak — whole-process resident set (`process_rss`) or CUDA allocator (`device_allocator`). The two are not on the same scale.

Key observation: **Parameters reproduce** for every model measured twice, while peak memory is reported by two different backends and is not comparable across the two machines.

Table 10: Memory overhead testing results on Small Machine

Model	Parameters (M)	Peak Memory (MB)
EfficientAD	8.06	1001.19
Faster R-CNN	41.35	610.70
GANomaly	188.69	2664.77
PaDiM	2.78	748.84
PatchCore	24.86	2498.00
STFPM	5.57	2767.25
SuperSimpleNet	33.72	1035.28
UNet++	53.97	1190.22
YOLO11n	2.59	2176.80
YOLOv8n	3.01	1095.68
YOLOv8s	11.14	2056.80

Table 11: Memory overhead testing results on Full Machine

Model	Parameters (M)	Peak memory (MB)	Allocator peak (MB)	Retained / extra (MB)	Memory backend
EfficientAD	8.0586	1869.2	1869.2	71.72	process_rss
Faster R-CNN	41.3523	588.9	588.9	314.74	device_allocator
GANomaly	188.6895	3044.3	3044.3	2159.62	process_rss
PaDiM	2.7828	2247.4	2247.4	168.49	process_rss
PatchCore	24.8625	4505.1	4505.0	455.11	process_rss
STFPM	5.5656	1879.1	1879.0	31.97	process_rss
SuperSimpleNet	33.7194	2365.4	2365.4	196.50	process_rss
UNet++	26.1934	535.0	535.0	200.05	device_allocator
YOLO11n	2.5900	65.5	65.5	5.23	device_allocator
YOLOv8n	3.0110	46.5	46.5	5.97	device_allocator
YOLOv8s	11.1360	121.5	121.5	21.48	device_allocator

Table 10 reports only the parameter count and the whole-process peak, so it has no allocator, retention or backend columns.

2.3 Analysis

The 19 configurations split into two paradigms: **supervised Defect Detection (DD)** and **unsupervised / zero-shot Anomaly Detection (AD)**. Both are scored on the metric families of [README §3.3](#) — image level, pixel level, instance level and overhead — and the rankings below use the Full Machine tables (Tables 3, 5 and 7).

Because the metrics have different scales, each model is reduced to one composite figure: its **mean rank** across every technical metric it reports, where 1 is best. The mean score column averages the same raw values.

Table 12: Anomaly detection — mean rank across the eight image- and pixel-level metrics

Model	Mean rank	Mean score	Metrics scored
MoECLIP	1.75	0.8851	8
PatchCore	2.38	0.8447	8
Dinomally	3.50	0.8311	8
WinCLIP	5.12	0.7270	8
EfficientAD	5.50	0.7369	8
STFPM	5.62	0.7795	8
PaDiM	6.00	0.6790	8
GANomaly	7.75	0.5958	4
Reverse Distillation	7.88	0.4959	8
SuperSimpleNet	8.38	0.4236	8

Table 13: Supervised detection — mean rank across the six instance-level metrics

Model	Mean rank	Mean score
Cascade R-CNN	1.83	0.5417
Faster R-CNN	2.17	0.5252
YOLO11n	3.33	0.4735
YOLOv8n	3.33	0.4668
YOLOv8s	4.33	0.4112
DETR	6.00	0.0006

- **Anomaly detection separates into three tiers.** MoECLIP (1.75), PatchCore (2.38) and Dinomally (3.50) are the usable detectors; WinCLIP to PaDiM (5.12–6.00) form a middle band; GANomaly, Reverse Distillation and SuperSimpleNet (7.75–8.38) are not usable. GANomaly is ranked on the four image metrics only, because it emits no pixel map.
- **Supervised detection is led by the two-stage detectors.** Cascade R-CNN (1.83) and Faster R-CNN (2.17) rank first; YOLO11n and YOLOv8n tie at 3.33 with YOLOv8s behind; DETR is degenerate (6.00). Segmentation ranks Mask R-CNN (0.7242 Dice) above UNet++ (0.6833) and DeepLabV3+ (0.6477).
- **Image level.** PatchCore (0.9932), MoECLIP (0.9877), Dinomally (0.9797) and WinCLIP (0.9772) are the sensitive

detectors; GANomaly, Reverse Distillation and SuperSimpleNet rank near chance.

- **Pixel level.** MoECLIP leads on localization (0.9857 AUROC, 0.9701 AUPRO), with PatchCore and Dinomaly close behind. Pixel F1 and IoU stay low for every anomaly model, whose pixel threshold is not calibrated.
- **Instance level.** Localization quality follows the box metrics: two-stage detectors first, YOLO trading recall for precision, DETR unusable.
- **Overhead.** Efficiency spans three orders of magnitude: YOLOv8n and YOLO11n are real-time, Faster R-CNN and PaDiM sit near 75 FPS, and PatchCore, EfficientAD, MoECLIP and WinCLIP sit at 1–5 FPS with 1.8–4.5 GB of memory.

Reproducibility. Core accuracy reproduces across both platforms — Image AUROC agrees within 0.2 points for all ten anomaly detectors, and parameter counts and FLOPs are exact. Efficiency overhead does not: FPS and latency are host-specific and differ by $1.1\times$ – $31\times$, and peak memory is reported by two different backends and is not comparable across machines.

3. Extensibility

3.1 Adding a New Dataset

Following [README §6.1](#), the `bottle` category of MVTec AD was copied into a new dataset root, `datasets/general/Bottle`, and PatchCore was trained and tested on it.

Table 14: Extensibility validation — PatchCore on the newly added Bottle dataset

Image AUROC	Pixel AUROC	Pixel AUPRO	IAP
1.0000	0.9935	0.9934	0.8461

The results are usable, confirming that the new dataset was added successfully and that PatchCore trained on it.

3.2 Adding the YOLO 26 Model

Following [README §6.2](#), YOLO 26 was added to the `ultralytics` backend:

- **Preset entry** — `yolo26n` variant and alias in `presets.py`
- **Model configuration** — `configs/models/ultralytics_yolo26_example.yaml`
- **Registry row** — one `yolo26n` entry in `configs/registry/models.yaml`
- **Training run** — ZJU-Leaper, test mode (a pipeline check, not an accuracy run): 1 epoch, batch 2, 640×640 input, 8 training and 8 validation samples

The run produced a new model, registered as `textile/artifacts/models/yolo26n_yolo26n_zju_leaper.pt` with 2.50 M parameters, confirming that YOLO 26 was added successfully.

4. Discussion

- **Training budgets are short.** Several entries were trained for only a few steps — the anomaly smoke configuration is 1 epoch on 8 images — so the weaker results in §2.3 reflect the run budget as much as the architecture.
- **Overhead metrics are host-defined.** FPS, latency, peak memory and stream counts are measured per machine and per profiling backend, so their absolute values and their meaning differ between the RTX 4090 and the A100 runs; only FLOPs and parameter counts are host-independent.
- **Coverage gaps.** MambaAD has not been tested; MoECLIP was scored on MVTec AD rather than ZJU-Leaper; cross-domain transfer ([README §3.3.4](#)) is covered by the Small Machine results only; and the compute tables omit `LMEI`, `Max streams @budget`, `1-stream latency`, `resolution slope` and power/energy, which need the resolution

sweep and a power-readable host.